# CSCI 4100

## Operating Systems and Architecture

Chapter 3: The Programming Interface

# Topics

- 3.0 – The Programming Interface
- 3.1 – Process Management
- 3.2 – Input/Output
- 3.3 – Case Study: Implementing a Shell
- 3.4 – Case Study: Interprocess Communication
- 3.5 – Operating System Structure

# The Programming Interface – The What

What functionality does an OS need to provide to applications?

- Process management
- Input/Output
- Thread management
- Memory management
- File systems and storage
- Networking and distributed systems
- Graphics and window management
- Authentication and security

# The Programming Interface – The Where

Where can we put this functionality?

- In a user-level program
- In a user-level library linked with each application
- In the OS kernel, accessed through a system call
- In a separate server process accessed via a system call

# The Programming Interface – The How

How do we make this choice?

- Flexibility: keep the system call interface simple to allow the software and hardware to be independent.
- Safety: resource management and protection must be implemented by the kernel.
- Reliability: reduce bugs in the kernel by keeping it small.
- Performance: transferring control to the kernel is expensive.

# Process Management

- In batch systems, users submit jobs and the OS creates a process when it is ready to run a job.
- In modern systems, user programs can create and manage their own processes.
- The process creator is called the **parent process**.
- The process that is created is called the **child process**.

# The Shell

- A **shell** is a job control system with a command-line interface.
- The shell prompts for a command and waits for the user.
- The user enters a command, usually consisting of a command name followed by command-line arguments.
- The shell interprets the command, and requests that the operating system create a process to execute the command.
- When the command completes, the shell prompts for the next command.

# Windows Process Management

On Windows, a process is created using the `CreateProcess` system call, which does the following:

1. Creates and initializes the PCB.
2. Creates and initializes a new address space.
3. Loads the program into the address space.
4. Copies arguments into memory in the address space.
5. Initializes the program counter to start execution at the start procedure.
6. Informs the scheduler that the new process is ready to run.

# UNIX Process Management

- On UNIX systems there are two system calls used to create a process.
- The `fork` system call creates a complete copy of the parent process.
- The `exec` system call is called by the child process to load the executable file into memory, and start running it.
- `fork` takes no arguments and returns an integer indicating if the process is the parent or the child.
- `exec` takes two arguments: the name of the program to run, and an array of arguments to pass to the program.

# UNIX Process Management – `fork`

The UNIX `fork` does the following:

1. Creates and initializes the PCB.
2. Creates a new address space.
3. Initializes the address space with a copy of the entire contents of the address space of the parent.
4. Inherits the execution context of the parent.
5. Inform the scheduler that the new process is ready to run.
6. Returns `0` to the child process, and the process ID of the child to the parent.

# UNIX Process Management - `exec`

The UNIX `exec` does the following:

1. Loads the program into the current address space.
2. Copies the arguments into memory in the address space.
3. Initializes the program counter to start execution at the start procedure.

UNIX provides three more essential system calls for managing processes:

- `wait` waits for a child process to complete before continuing with the rest of the program.
- `signal` sends a message to a process using an upcall.
- `exit` tells the kernel that the calling process has completed and its resources can be deallocated.

# Input/Output (1)

- Early computer systems provided customized programming interfaces for each hardware device.
- UNIX and other modern operating systems provide a single interface for device I/O, file operations, and interprocess communication.
- All I/O uses the `open`, `close`, `read`, and `write` system calls.
- The `open` system call must be used first and returns a **file descriptor** that can be used by subsequent `read` and `write` operations.
- All data is stored in arrays of bytes.

# Input/Output (2)

- Data being read is stored in a kernel buffer. If the buffer is empty, `read` blocks the caller.
- Data being written is also stored in a kernel buffer. If the buffer is full, `write` blocks the caller.
- The `close` system call signals that a process is done with the file, device, or communication channel.
- A **pipe** is a kernel buffer that can be written to by one process and read from by another process. The `pipe` system call creates a pipe.
- The open file descriptors of a process can be replaced using the `dup2` system call.
- The `select` system call waits for input from an array of file descriptors.

# Case Study: Implementing a Shell (1)

- Every UNIX process has three open file descriptors when it starts running.
- File descriptor 0 represents **standard input**.
- File descriptor 1 represents **standard output**.
- File descriptor 2 represents **standard error**.
- By default, all of these file descriptors are connected to the console that launched the process.

# Case Study: Implementing a Shell (2)

The shell processes a command line as follows:

1. The shell reads and parses the command line to extract the command and its arguments.
2. The shell calls `fork` to create a child process with copies of the same file descriptors as its parent.
3. The child process calls `exec` to execute the command with the provided arguments.
4. The parent process calls `wait` to wait until the child process completes.

# Case Study: Implementing a Shell (3)

- A UNIX process does not know where its input is coming from or where its output is going.
- A process can have its output redirected to a file.
- A process can have its input redirected from a file.
- One process can write to a pipe, and another process can read from that pipe.
- Multiple processes can use pipes to set up a **pipeline**.
- A **shell script** is a file containing a sequence of commands for the shell to execute.

# Case Study: Interprocess Communication

Processes can communicate with each other in a variety of ways.

- A **producer** can generate output, which is read and processed by a **consumer**.
- A **client** can send a request to a **server**, which sends a response back to the client.
- Processes can communicate using files in the file system.

# The Producer-Consumer Model

Producer-consumer communication works as follows:

1. The OS creates a pipe to use for communication.
2. The producer's output is redirected to the pipe's input. Writes are blocked if the pipe is full.
3. The consumer's input is redirected to the pipe's output. Reads are blocked if the pipe is empty.
4. When the producer finishes, it closes its end of the pipe.
5. When the consumer reads an end-of-file marker from the pipe, it closes its end of the pipe.

# The Client-Server Model

- Client-server communication is similar, except two pipes are required.
- The client sends requests to the first pipe, and reads responses from the second pipe.
- The server reads requests from the first pipe, and sends responses to the second pipe.
- Additional pipes can be created if there are multiple clients.

# Operating System Structure

- There are many dependencies between the modules of an OS.
- Many modules use synchronization primitives to coordinate access to shared data structures.
- The virtual memory system depends on hardware support for address translation.
- The file system and virtual memory system depend on physical memory and the disk device driver.
- The file system may depend on the network protocol stack to access a disk on a remote machine.

# Monolithic Kernels

A **monolithic kernel** implements most OS functionality inside the kernel.

- A **hardware abstraction layer (HAL)** provides a common interface to a variety of processors.
- **Dynamically loadable device drivers** decouple the OS code from specific I/O devices.
- Linux, BSD, DOS, and early versions of Windows use a monolithic kernel design.

# Microkernels

A **microkernel** offloads as much functionality as it can to user-level servers.

- Window managers, the network stack, and file systems can be implemented at the user level.
- The kernel acts as an intermediary between client applications and these servers.
- Microkernels make development easier, but add additional overhead.
- Windows and MacOS use a hybrid model with some service in the kernel, and others at user-level.